

Source:

backend/src/controllers/registro.controller.js

Home

Namespaces

apiConfig

contactoEmergenciaService

diarioService

Classes

ErrorBoundary

Events

SIGINT

unhandledRejection

Global

404_Error_Handler

ACTIVIDADES

API_URL

ASPECTOS_COGNITIVOS

ActionCard

ActivitySelector

App

AuthButtons

AuthLayout

Button

CORS_Configuration

Card

Carousel

Checkbox

CircularProgress

CognitionSelector

DiaryBanner

DiaryEditor

DiaryEntry

EMOCIONES

EmergencyButton

EmergencyModal

EmotionSelector

EmotionStats

Footer

GET_/

Generic_Error_Handler

Heading

HeroSection

Home

Input

Landing

LandingHero

Login

Logo

MainLayout

Navbar

ProtectedRoute

Register

Seguimiento

Slider

```
1.  const Registro = require('../models/registro_mongoose');
2.
3.  /**
4.   * @function createRegistro
5.   * @description Crea un nuevo registro diario del usuario
6.   * @param {object} req - Request object
7.   * @param {object} res - Response object
8.   */
9.  const createRegistro = async (req, res) => {
10.    try {
11.      console.log('  createRegistro called');
12.      console.log('User from middleware:', req.user); // DEB
13.
14.      if (!req.user || !req.user.id) {
15.        console.error('  User not found in request');
16.        return res.status(401).json({ message: 'User not a
17.      }
18.
19.      const usuarioId = req.user.id;
20.      const registroData = req.body;
21.
22.      console.log('Creando registro para usuario:', usuarioId);
23.      console.log('Datos del registro:', Object.keys(registroData));
24.
25.      const nuevoRegistro = new Registro({
26.        ...registroData,
27.        usuarioId,
28.      });
29.
30.      await nuevoRegistro.save();
31.
32.      console.log('  Registro guardado exitosamente:', nuevoRegistro);
33.      res.status(201).json({ message: 'Registro diario guardado exitosamente' });
34.    } catch (error) {
35.      console.error('  Error creating registro:', error);
36.      res.status(500).json({ message: 'Error al guardar el registro' });
37.    }
38.  };
39.
40.  /**
41.   * @function getRegistros
42.   * @description Obtiene todos los registros del usuario autenticado
```

```

43.  * @param {object} req - Request object
44.  * @param {object} res - Response object
45.  */
46.  const getRegistros = async (req, res) => {
47.    try {
48.      console.log('  getRegistros called');
49.      console.log('User from middleware:', req.user); // DEB
50.
51.      if (!req.user || !req.user.id) {
52.        console.error('  User not found in request');
53.        return res.status(401).json({ message: 'User not a
54.      }
55.
56.      const usuarioId = req.user.id;
57.
58.      console.log('Obteniendo registros para usuario:', usua
59.      const registros = await Registro.find({ usuarioId: usu
60.
61.      console.log('  Se encontraron', registros.length, 're
62.      res.status(200).json({ data: registros });
63.    } catch (error) {
64.      console.error('  Error getting registros:', error);
65.      res.status(500).json({ message: 'Error al obtener los
66.    }
67.  };
68.
69.  /**
70.   * @function getRegistroById
71.   * @description Obtiene un registro específico por su ID
72.   * @param {object} req - Request object
73.   * @param {object} res - Response object
74.   */
75.  const getRegistroById = async (req, res) => {
76.    try {
77.      console.log('  getRegistroById called with ID:', req.I
78.
79.      if (!req.user || !req.user.id) {
80.        console.error('  User not found in request');
81.        return res.status(401).json({ message: 'User not a
82.      }
83.
84.      const { id } = req.params;
85.      const usuarioId = req.user.id;
86.
87.      console.log('Buscando registro:', id, 'para usuario:',
88.      const registro = await Registro.findOne({ _id: id, usu
89.
90.      if (!registro) {
91.        console.log('⚠ Registro no encontrado');

```

Text

Textarea

accessWithPassword

actualizarEntradaDiario

api

apiFetch

authMiddleware

closeShareModal

compararPassword

componentDidCatch

connectDBAndStartServer

copyShareLink

crearEntradaDiario

create

createContacto

createRegistro

delete

deleteContacto

description

dismiss

duracion

eliminarEntradaDiario

emotion

error

formatDate

formatDateShort

getAll

getById

getContactoById

getContactos

getDerivedStateFromError

getIntensidad

getPreviewText

getProfile

getRegistroByFecha

getRegistroById

getRegistros

getRegistrosByRango

getStepLabel

goToNext

goToPrevious

handleAddActividad

handleAddContact

handleCall

handleCancelAdd

handleCancelEditor

handleChange

handleCreateEntry

handleDeleteEntry

handleEditEntry

handleEmocionToggle

handleInputChange

handleIntensidadChange

handleLogin

handleLogout

handleOverlayClick

handleRegister

handleReload

handleRemoveActividad

```

92.         return res.status(404).json({ message: 'No se enco
93.     }
94.
95.     console.log('    Registro encontrado');
96.     res.status(200).json({ data: registro });
97. } catch (error) {
98.     console.error('    Error getting registro by ID:', erro
99.     res.status(500).json({ message: 'Error al obtener el r
100. }
101. };
102.
103. /**
104.  * @function getRegistroByFecha
105.  * @description Obtiene registros de una fecha específica
106.  * @param {object} req - Request object
107.  * @param {object} res - Response object
108.  */
109. const getRegistroByFecha = async (req, res) => {
110.     try {
111.         console.log('    getRegistroByFecha called for:', req.p
112.
113.         if (!req.user || !req.user.id) {
114.             console.error('    User not found in request');
115.             return res.status(401).json({ message: 'User not a
116.         }
117.
118.         const { fecha } = req.params;
119.         const usuarioId = req.user.id;
120.
121.         const startOfDay = new Date(fecha);
122.         startOfDay.setUTCHours(0, 0, 0, 0);
123.
124.         const endOfDay = new Date(fecha);
125.         endOfDay.setUTCHours(23, 59, 59, 999);
126.
127.         const registros = await Registro.find({
128.             usuarioId: usuarioId,
129.             fechaCreacion: {
130.                 $gte: startOfDay,
131.                 $lte: endOfDay,
132.             },
133.         }).sort({ fechaCreacion: -1 });
134.
135.         if (!registros || registros.length === 0) {
136.             console.log('⚠ No registros found for date:', fech
137.             return res.status(404).json({ message: 'No se enco
138.         }
139.
140.         console.log('    Se encontraron', registros.length, 're

```

```

handleSendEmail
handleSendSms
handleShareEntry
handleSubmit
handleToggle
handleToggleExpand
handleUpdateEntry
info
intensidad
isMobileDevice
isSelected
loadContacts
loadEntries
loading
loginUser
nextStep
nombre
obtenerEntradaDiarioPorId
obtenerEntradasDiario
optionalAuthMiddleware
percentage
port
pre-save
prevStep
promise
registerUser
render
request-interceptor
response-interceptor
sendEmergencyEmail
success
timeAgo
title
togglePasswordField
update
updateContacto
uri
useAuthStore
useToast
validateRequiredEnvVars

```

```

141.         res.status(200).json({ data: registros });
142.     } catch (error) {
143.         console.error('    Error getting registros by fecha:', error);
144.         res.status(500).json({ message: 'Error al obtener los registros' });
145.     }
146. };
147.
148. /**
149.  * @function getRegistrosByRango
150.  * @description Obtiene registros en un rango de fechas
151.  * @param {object} req - Request object
152.  * @param {object} res - Response object
153.  */
154. const getRegistrosByRango = async (req, res) => {
155.     try {
156.         console.log('    getRegistrosByRango called');
157.
158.         if (!req.user || !req.user.id) {
159.             console.error('    User not found in request');
160.             return res.status(401).json({ message: 'User not authenticated' });
161.         }
162.
163.         const { fechaInicio, fechaFin } = req.query;
164.         const usuarioId = req.user.id;
165.
166.         if (!fechaInicio || !fechaFin) {
167.             console.log('⚠ Fechas no proporcionadas');
168.             return res.status(400).json({ message: 'Se requieren fechas de inicio y fin' });
169.         }
170.
171.         const start = new Date(fechaInicio);
172.         start.setUTCHours(0, 0, 0, 0);
173.
174.         const end = new Date(fechaFin);
175.         end.setUTCHours(23, 59, 59, 999);
176.
177.         console.log('Buscando registros entre:', start, 'y', end);
178.         const registros = await Registro.find({
179.             usuarioId: usuarioId,
180.             fechaCreacion: {
181.                 $gte: start,
182.                 $lte: end,
183.             },
184.         }).sort({ fechaCreacion: -1 });
185.
186.         console.log('    Se encontraron', registros.length, 'registros');
187.         res.status(200).json({ data: registros });
188.     } catch (error) {
189.         console.error('    Error getting registros by rango:', error);

```

```
190.         res.status(500).json({ message: 'Error al obtener los
191.     }
192. };
193.
194. module.exports = {
195.     createRegistro,
196.     getRegistros,
197.     getRegistroById,
198.     getRegistroByFecha,
199.     getRegistrosByRango,
200. };;
```